

Formal Agentic Specification and Prompting Doctrine

Version 1.0.1

Golden Master Release with Adaptive Doctrine Enrichment

August 25, 2026

Abstract

This document defines a formal specification for deterministic multi-agent orchestration. Version 1.0.1 integrates the core doctrine of Version 1.0.0 with all amendments from RFC-2026-08 and the Adaptive Doctrine Enrichment (ADE) addendum, including formal remediations from the peer review. It establishes the Dual-Layer Abstraction Principle, multi-source epistemic oracles with snapshot anchoring, Beacon-based workspace concurrency, linear-time secret scanning, CVCP subgraph automata with refined dynamic budgets, quantized context compaction, and a controlled self-improvement mechanism (ADE) that detects and fills gaps in style guides and RAG knowledge while preserving determinism, safety, and budget constraints.

Document Status and Provenance. Version 1.0.1 is a golden-master consolidation. The core doctrine sections (§2–§10) are carried forward unchanged from the prior master release and are intentionally *not restated* in this document (see Section 2); their invariants remain binding and are cited throughout. The Adaptive Doctrine Enrichment (ADE) layer is new in this release. RFC-2026-08 is referenced as an external amendment record; it is not reproduced here.

Contents

1	Introduction and Dual-Layer Abstraction	2
1.1	Dual-Layer Architectural Doctrine	2
1.2	Formal Lowering Function	2
2	Consolidated Core Doctrine (Sections 2–10)	2
3	Adaptive Doctrine Enrichment and Background Execution	3
3.1	Overview	3
3.2	New Primitives and Their Formal Semantics	3
3.2.1	Emission versus Mutation	3
3.2.2	Tool Grants	3
3.3	Gap Detection and Proposal Generation	3
3.4	Curation and Validation	4
3.5	Background Subprocesses and Budget Separation	4
3.6	Epoch-Gated Promotion and Tier-Gated Escalation	5
3.7	Doctrine Pruning and Anti-Bloat	5
3.8	Formal Integration with Existing Invariants	6
3.9	Extended Conformance Tests	6
4	Conclusion	6

1 Introduction and Dual-Layer Abstraction

1.1 Dual-Layer Architectural Doctrine

The specification separates formal mathematical abstraction from concrete engine implementation.

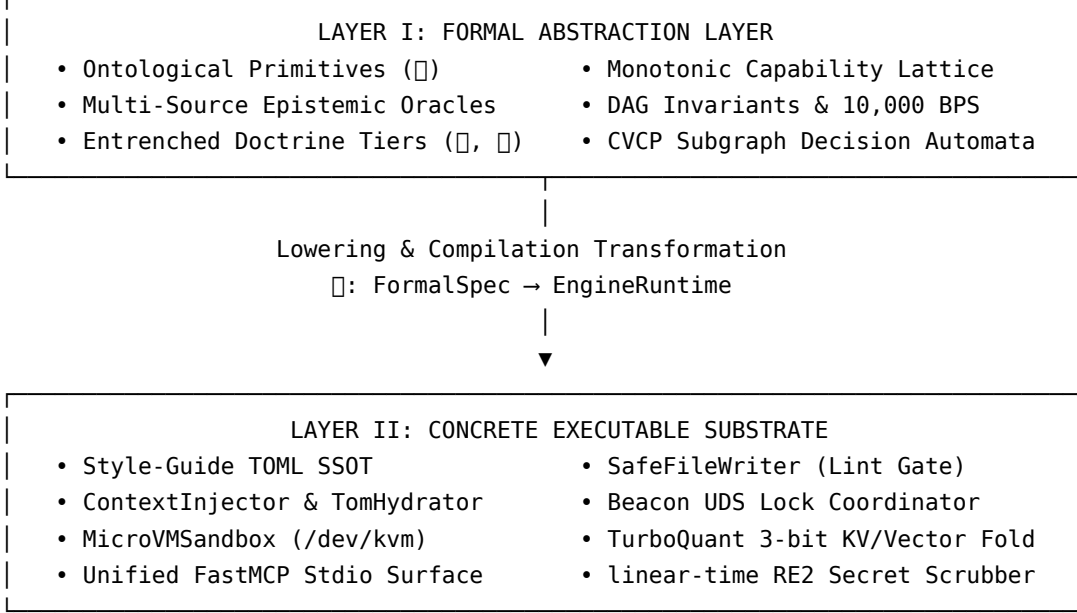


Figure 1: Dual-Layer Abstraction Principle

1.2 Formal Lowering Function

The lowering function is a four-stage pipeline. Stage 1 merges tier-lowered rule fragments; stages 2–3 render and canonicalize. The merge — not the canonicalizer — owns conflict resolution and tier precedence.

$$\mathcal{R}_{\text{merged}} = \text{MergeRules}(\text{Lower}(T_1), \text{Lower}(T_2(\sigma)), \text{Lower}(T_4), \text{Inject}_{T_3}(e), \text{Hydrate}(\mathcal{M}_{\text{mem}})) \quad (1)$$

$$\mathcal{X} = \text{RenderXML}(\mathcal{R}_{\text{merged}}) \quad (2)$$

$$(\sigma, e, \tau, \mathcal{M}_{\text{mem}}) = \text{C14N_XML}(\mathcal{X}) \quad (3)$$

MergeRules is a deterministic total function: fragment union with implied-repeal resolution ordered by tier precedence ($T_1 \succ T_2 \succ T_3 \succ T_4$), ties broken by rule identifier (lexicographic). **Inject** _{T_3} and **Hydrate** produce *candidate* fragments only; candidates become part of $\mathcal{R}_{\text{merged}}$ solely through **MergeRules**, never by bypassing it.

2 Consolidated Core Doctrine (Sections 2–10)

Sections 2–10 — ontological primitives, multi-source epistemic oracles with snapshot anchoring, the entrenched doctrine tiers ($\mathbb{T}, \preceq$) and their amendment protocol, the monotonic capability lattice, DAG invariants and basis-point budgets, CVCP subgraph decision automata, quantized context compaction, linear-time secret scanning, and the MicroVM sandbox — are carried forward *verbatim* from the prior master release and are deliberately not restated in this consolidation. Their invariants remain binding; the ADE layer (Section 3) is defined strictly as an extension of, and is constrained by, those invariants.

3 Adaptive Doctrine Enrichment and Background Execution

3.1 Overview

Version 1.0.1 extends the specification with a self-improving mechanism: Adaptive Doctrine Enrichment (ADE) and background subprocesses. The system can now detect gaps in its own doctrine (style guides, RAG knowledge) and propose remediations that are validated by the existing epistemic oracle and verification pipeline, then promoted only at safe epoch boundaries.

3.2 New Primitives and Their Formal Semantics

The primitive fleet is extended:

$$\mathcal{P}' = \mathcal{P} \cup \{\mathbf{observe}, \mathbf{enrich}, \mathbf{curate}\}$$

These primitives are strictly non-mutating with respect to $\mathcal{A}_{\text{workspace}}$:

$$\mathbf{Mut}(\mathbf{observe}) = \emptyset, \quad \mathbf{Mut}(\mathbf{enrich}) = \emptyset, \quad \mathbf{Mut}(\mathbf{curate}) = \emptyset$$

They operate on the extended artifact space:

$$\mathcal{A}'_{\text{artifact}} = \mathcal{A}_{\text{artifact}} \cup \{\mathcal{S}_{\text{gaps}}, \mathcal{S}_{\text{proposals}}, \mathcal{S}_{\text{curated_patch}}\}$$

Emission functions:

$$\mathbf{Emit}(p) = \begin{cases} \{\mathcal{S}_{\text{gaps}}\}, & p = \mathbf{observe} \\ \{\mathcal{S}_{\text{proposals}}\}, & p = \mathbf{enrich} \\ \{\mathcal{S}_{\text{curated_patch}}\}, & p = \mathbf{curate} \end{cases}$$

Capability class mapping:

$$\mathbf{ClassOf}(\mathbf{observe}) = \mathbf{verifier}, \quad \mathbf{ClassOf}(\mathbf{enrich}) = \mathbf{reasoner}, \quad \mathbf{ClassOf}(\mathbf{curate}) = \mathbf{verifier}$$

3.2.1 Emission versus Mutation

$\mathbf{Mut}(p) = \emptyset$ asserts that no new primitive is ever granted **exec**-equivalent authority over $\mathcal{W}_{\text{workspace}}$. Emission is a distinct, weaker operation:

$$\mathbf{Persist} : \mathcal{P}' \times \mathcal{A}'_{\text{artifact}} \rightarrow \mathcal{A}'_{\text{artifact}} \setminus \mathcal{W}_{\text{workspace}}, \quad \mathbf{range}(\mathbf{Persist}) \cap \mathcal{W}_{\text{workspace}} = \emptyset$$

$\mathbf{Emit}(p) = S$ denotes **Persist** of an immutable, content-addressed record into the staging artifact space. Staged artifacts are inert: they never enter the active prompt frame or doctrine build until promoted by the epoch-gated mechanism of Section 3.6.

3.2.2 Tool Grants

Table 1: Static Tool Grants for New Primitives

Primitive	Static Grant $\mathcal{T}(p)$	Risk Ceiling
observe	file_read, code_search, rag_search, query_fold, web_search	LOW
enrich	rag_search, web_search, code_context, file_discovery	LOW
curate	file_read, code_search, code_context, graph_callers	LOW

3.3 Gap Detection and Proposal Generation

The **observe** primitive continuously analyzes artifacts and logs to detect doctrine gaps. A gap is defined by:

$$\mathbf{GapDetected}(c, \tau) \iff \exists \text{pattern} \in \mathbf{Patterns}(\tau) : \neg \mathbf{CoveredByDoctrine}(\text{pattern}, \mathbb{D}) \wedge \mathbf{Frequency}(\text{pattern}) \geq \theta_F$$

Upon detection, **observe** emits $\mathcal{S}_{\text{gaps}}$.

Operator Grounding (implementability contract). The gap predicate is decidable only once its operators are grounded:

- **Patterns**(τ): recurring tool-call and error signatures extracted from Beacon ring telemetry over analysis window τ (default 7 days), deduplicated by normalized signature hash.
- **Frequency**(pattern): occurrence count within τ , normalized by distinct session count; the pattern must appear in ≥ 2 distinct sessions.
- **CoveredByDoctrine**(pattern, $\mathbb{D}$): false iff a doctrine-corpus retrieval (`rag_search`, `top-k = 5`) returns no hit at or above similarity threshold θ_{sim} .
- Defaults: $\theta_{\text{freq}} \geq 3$, θ_{sim} fixed per embedding model card. All four operators are deterministic given the telemetry snapshot; `observe` pins the snapshot id so gap reports are replayable.

The `enrich` primitive consumes gap reports and generates proposals:

$$p = \langle \text{id}, \text{target_tier}, \text{target_scope}, \text{content}, \text{justification} \rangle$$

Each proposal must include citations to source material. Uncited proposals are labeled **HYPOTHESIS** and cannot be promoted without further evidence.

3.4 Curation and Validation

The `curate` primitive validates staged proposals against four gates:

1. **Citation Verification:** Proposals must pass the 3-level citation oracle (trace inspection, boundary check, verbatim match).
2. **Conflict Detection:** No conflict with existing rules after implied repeal resolution.
3. **Entrenchment Preservation:** If targeting T_1 or T_4 , the proposal must be a restriction (narrowing), not widening.
4. **Budget Ceiling:** The byte size of the proposed addition must fit within the tier budget, i.e. $\|\text{content}\|_{\text{bytes}} \leq \mathbf{Cap}(T_i)$ for target tier T_i , where **Cap** is the tier byte-budget function defined in the core budgets section (default 2048 bytes for T_3 extensions).
5. **Source Trust Tiering:** Every citation must resolve to a source in a trusted origin class (first-party repository files, first-party documentation). Web-sourced citations are admissible only if (i) two independent origins agree on the cited content, and (ii) any proposal targeting T_2 or above is additionally flagged for human review at promotion. This closes the soundness hole where attacker-controlled web content passes verbatim-match verification while being untrustworthy.

3.5 Background Subprocesses and Budget Separation

Background processes run as daemons, separate from the main DAG. They form a set $\mathcal{B}$ with their own budget envelope.

Dual-Envelope Budget Conservation (Remediation 1): Let π be the fixed unit-pricing schedule (USD per token class), published before any run. The envelopes are USD allocations:

$$E_{\text{main}} + E_{\text{bg}} \leq \text{Budget}_{\text{hard_limit_usd}}$$

Each envelope is decomposed internally as a basis-point partition over its own vertices, and realized cost is priced through π :

$$\begin{aligned} \sum_{v \in V_{\text{main}}} \beta_{\text{main}}(v) &= 10000 \text{ BPS}, & \sum_{v \in V_{\text{main}}} \pi(\text{usage}(v)) &\leq E_{\text{main}} \\ \sum_{b \in \mathcal{B}} \beta_{\text{bg}}(b) &\leq 10000 \text{ BPS}, & \sum_{b \in \mathcal{B}} \pi(\text{usage}(b)) &\leq E_{\text{bg}} \end{aligned}$$

Basis points express *relative priority within an envelope*; dollars express *absolute spend against that envelope*. Exhausting E_{bg} halts background daemons only; it never deducts from E_{main} (TC-12), and vice versa.

3.6 Epoch-Gated Promotion and Tier-Gated Escalation

Epoch-Gated Atomic Promotion (Remediation 2): Proposals are promoted only at workflow boundaries when **ActiveRuns** = 0, under a Beacon lock, with re-compilation of the doctrine and emission of a new hash.

Promotion Liveness (lease-based locks): the Beacon lock is a *lease*, not a bare mutex: it carries a TTL with holder heartbeat. A promoter that dies mid-promotion releases the lock by TTL expiry; any peer may then take over after expiry without manual intervention, so a crashed promotion can delay but never permanently deadlock doctrine evolution. Abandoned partial promotions are rolled back from the pre-promotion hash before takeover.

Serialized Batch Application: all proposals validated since the last promotion are applied as one serialized batch under a single lock acquisition. Two proposals whose target scopes overlap conflict unless one strictly dominates the other by tier precedence; conflicting equal-tier proposals are both denied (fail-closed) and returned to **enrich** with the conflict reason. Silent last-writer-wins is prohibited.

Doctrine Compilation. Promotion concludes by compiling the merged doctrine into the active artifact set; compilation enforces all byte-cap gates (**Cap**(T_i) per tier) and fails closed on violation. This compilation step is what conformance test TC-15 exercises.

Mandatory Human Escalation for T1/T4 (Remediation 3): Any proposal targeting T_1 or T_4 triggers **ESCALATE** and requires authenticated human signature before promotion.

3.7 Doctrine Pruning and Anti-Bloat

AutoDream Pruning Operator (Remediation 4): To respect byte caps, a pruning operator compacts low-utility rules:

$$\text{Prune}_{\text{doctrine}}(T_i) = \begin{cases} T_i, & \|T_i\|_{\text{bytes}} \leq \mathbf{Cap}(T_i) \\ \mathbf{CompressAndDeduplicate}(\mathbf{SortByUtility}(T_i)), & \|T_i\|_{\text{bytes}} > \mathbf{Cap}(T_i) \end{cases}$$

where **Cap**(T_i) is the tier byte-budget function from the core budgets section (2048 bytes is the T_3 default).

Utility and Determinism. Rule utility is $U(r) = \alpha \cdot \text{matches}(r, W) - \delta \cdot \|r\|_{\text{bytes}}$, where $\text{matches}(r, W)$ counts invocation-telemetry hits over lookback window W (default 30 days, sourced from Beacon invocation records). **SortByUtility** orders by U descending with rule-identifier lexicographic order as total-order tie-breaker, so pruning output is a deterministic function of the telemetry snapshot id. **CompressAndDeduplicate** removes rules whose normalized content hashes collide (keeping the highest-utility instance), then replaces each surviving low-utility rule body with a content-addressed TurboQuant fold pointer retrievable via **query_fold**; archived bodies remain auditably recoverable and never silently vanish. Pruned rules that receive no hits across two consecutive windows are deleted; all others are archived.

3.8 Formal Integration with Existing Invariants

The new primitives and background processes preserve:

- Mutation isolation: only `exec` writes workspace.
- Monotonic tool grants.
- DAG acyclicity (background processes are a separate acyclic graph).
- Dual budget conservation.
- Entrenchment with human gate for T1/T4.

3.9 Extended Conformance Tests

Table 2: Additional Conformance Tests (TC-10 to TC-15)

Test	Invariant	Assertion
TC-10	Main flow non-blocking	Background observe/enrich crashes do not interrupt active DAG.
TC-11	No staging auto-merge	Unvalidated proposals never appear in active prompt frame.
TC-12	Dual-envelope budget isolation	Background budget exhaustion does not deduct from main DAG.
TC-13	Human gate on T1/T4	Proposals targeting T1/T4 halt at ESCALATE, reject autonomous
TC-14	Epoch-gated promotion	Doctrine TOML updates blocked while <code>ActiveRuns</code> > 0.
TC-15	Auto-pruning ceiling	When $\ T_3(e)\ > \mathbf{Cap}(T_3)$, AutoDream GC compacts below cap before
TC-16	Serialized epoch application	Conflicting equal-tier proposals in one batch are both denied; no sil
TC-17	Curate staging isolation	Every Persist performed by <code>curate</code> lands in $\mathcal{A}'_{\text{artifact}} \setminus \mathcal{W}_{\text{workspace}}$; w
TC-18	Source-trust gate	A proposal citing a single untrusted web origin is denied at gate 5 c

4 Conclusion

Version 1.0.1 merges the formal agentic doctrine with a self-improving adaptive layer. The system now not only executes deterministic workflows with fail-closed security and verifiable grounding, but also continuously improves its own domain knowledge and rule set under strict mathematical control. The dual-envelope budget model, epoch-gated promotion, human escalation for entrenched tiers, and automatic pruning ensure that autonomous learning does not compromise determinism, safety, or cost predictability. Conforming orchestrators achieve a new level of robustness: a system that can evolve while remaining auditably correct.